

Just make it

```
(fighting@kali)-[~/桌面/os_labcode/lab1]
$ make qemu
+ cc kern/init/entry.S
+ cc kern/init/init.c
+ cc kern/libs/stdio.c
+ cc kern/driver/console.c
+ cc libs/printfmt.c
+ cc libs/readline.c
+ cc libs/sbi.c
+ cc libs/string.c
+ ld bin/kernel
riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img

OpenSBI v1.6

Platform Name      : riscv-virtio,qemu
Platform Features  : medeleg
Platform HART Count : 1
Platform IPI Device : aclint-mswi
Platform Timer Device : aclint-mtimer @ 10000000Hz

Boot HART Priv Version : v1.12
Boot HART Base ISA      : rv64imafdc
Boot HART ISA Extensions : sstc,zicntr,zihpm,zicboz,zicbom,sdtrig,svadu
Boot HART PMP Count     : 16
Boot HART PMP Granularity : 2 bits
Boot HART PMP Address Bits : 54
Boot HART MHPM Info     : 16 (0x0007fff8)
Boot HART Debug Triggers : 2 triggers
Boot HART MIDELEG       : 0x0000000000001666
Boot HART MEDELEG       : 0x0000000000f4b509
```

```
int kern_init(void) {
    extern char edata[], end[];
    // 这里声明的两个符号由链接器定义，分别指向.data段结束和.bss段结束
    memset(edata, 0, end - edata);
    // 清除.bss段：由于内核没有标准库，memset需要我们自己实现

    const char *message = "(THU.CST) os is loading ...\n";
    cprintf("%s\n\n", message); // cprintf是我们在ucore中自己实现的格式化输出函数

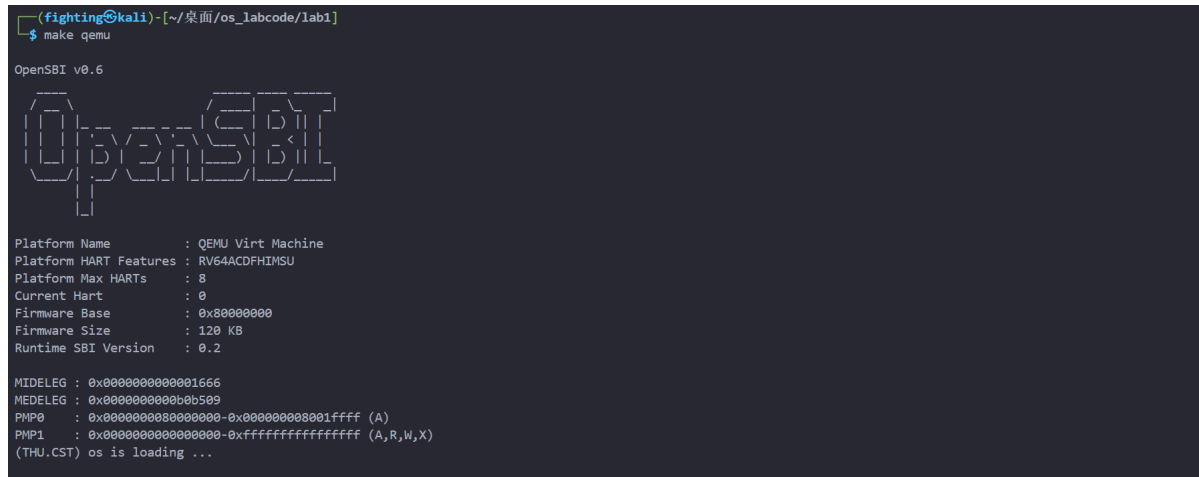
    while (1)
        ; // 进入无限循环
}
```

后发现是opensbi版本太高了，对于makefile中的最后一句，他只会加载到内存中，却不会自动从0x80200000之后开始执行代码。

```
.PHONY: qemu
qemu: $(UCOREIMG) $(SWAPIMG) $(SFSIMG)
# $(V)$(QEMU) -kernel $(UCOREIMG) -nographic
$(V)$(QEMU) \
    -machine virt \
    -nographic \
    -bios default \
    -device loader,file=$(UCOREIMG),addr=0x80200000
```

后由AI建议改为 `-kernel bin/ucore.img` 会自动加载默认地址0x80200000,然后自动执行,为了保持和指导书一致最终选择降级版本。

替换qemu中的opensbi为低版本,即可成功运行。



```
(fighting@kali) - [~/桌面/os_labcode/lab1]
$ make qemu

OpenSBI v0.6

Platform Name       : QEMU Virt Machine
Platform HART Features : RV64ACDFHIMSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base       : 0x80000000
Firmware Size       : 120 KB
Runtime SBI Version  : 0.2

MIDELEG : 0x0000000000001666
MEDELEG : 0x00000000000b509
PMP0    : 0x0000000000000000-0x000000000001ffff (A)
PMP1    : 0x0000000000000000-0xffffffffffffffff (A,R,W,X)
(THU.CST) os is loading ...
```

GDB调试

练习

练习1: 理解内核启动中的程序入口操作

阅读 `kern/init/entry.S` 内容代码, 结合操作系统内核启动流程, 说明指令 `la sp, bootstacktop` 完成了什么操作, 目的是什么? `tail kern_init` 完成了什么操作, 目的是什么?

`la sp, bootstacktop`

- 操作说明:

`la sp, bootstacktop` 是 RISC-V 汇编中的“Load Address”伪指令, 用于将符号 `bootstacktop` 所代表的内存地址加载到栈指针寄存器 `sp` 中。

该符号在链接脚本中被定义为内核启动栈 (`bootstack`) 的**栈顶地址**, 即栈空间的最高地址, 通常位于 `.data` 段中通过 `.space KSTACKSIZE` 预留的一块静态内存区域的末尾。

- 目的:

内核刚被 OpenSBI 跳转执行时, CPU 的栈指针处于未初始化状态。而后续调用 C 函数 (如 `kern_init`) 必须依赖有效的栈来保存返回地址、函数参数和局部变量。

因此, 该指令的核心目的是**为内核初始化阶段建立一个可用的运行时栈**, 确保 C 代码能够正常运行。

RISC-V 的栈向低地址增长, 故 `sp` 必须初始化为栈空间的高地址 (即 `bootstacktop`)。

tail kern_init

- **操作说明:**

`tail kern_init` 是 RISC-V 汇编中的**尾调用跳转指令**（等价于 `j kern_init`），它直接将程序计数器（PC）设置为 `kern_init` 函数的入口地址，并**不保存返回地址**（即不修改 `ra` 寄存器）。此时 `kern_init` 是用 C 语言编写的内核主初始化函数，其地址由链接器确定。

- **目的:**

将控制权从汇编编写的**内核入口点**（`kern_entry`）正式移交到**C 语言实现的内核初始化逻辑**，完成从底层引导代码到高级语言运行环境的过渡。

练习2: 使用GDB验证启动流程

为了熟悉使用 QEMU 和 GDB 的调试方法，请使用 GDB 跟踪 QEMU 模拟的 RISC-V 从加电开始，直到执行内核第一条指令（跳转到 `0x80200000`）的整个过程。通过调试，请思考并回答：RISC-V 硬件加电后最初执行的几条指令位于什么地址？它们主要完成了哪些功能？请在报告中简要记录你的调试过程、观察结果和问题的答案。

答: